Machine Learning Assignment

Topic:

Kernel Learning on Riemannian Manifold with Geodesic Distance Preservation

Student Name: Chandaka Preetesh

Registration Number: 23CSB0A47

April 30, 2026

Abstract

Here I used theoretical and empirical justification of the entire kernel learning pipeline on manifolds endowed with geodesic distance preservation. Data points—specifically SPD covariance matrices computed from images—are projected to a tangent space using the matrix logarithm. From there, we learn a supervised Mahalanobis metric using ITML. The initial kernel built from this learned metric is projected using Bregman projection to satisfy geodesic distances between points. To test our pipelines, we utilized two standard benchmarks: the Olivetti Faces collection, which contains 100 images and 336 texture samples from Brodatz dataset.

Contents

1 Paper Reference **3**

2 Problem Understanding and Importance **3**

2.1 Background and Motivation 3

2.2 Problem Statement 4

2.3 Importance 4

2.4 Dataset Description 4

2.4.1 Dataset 1: Olivetti Faces 5

2.4.2 Dataset 2: Brodatz Textures 5

3 Model Implementation **7**

3.1 Pipeline Overview 7

3.2 Step 1: Feature Extraction 7

3.3 Step 2: Covariance Matrix Computation (SPD) 8

3.4 Step 3: Log-Euclidean Mapping 9

3.5 Step 4: Vectorisation and Normalisation 10

3.6 Step 5: Geodesic Distance Computation 10

3.7 Step 6: Discriminative Distance Learning (Mahalanobis Distance ITML) . 11

3.8 Step 7: Initial Kernel Construction 12

3.9 Step 8: Bregman Kernel Optimisation 13

3.10 Step 9: Baseline Kernel Construction 14

3.10.1 Gaussian (RBF) Kernel 14

3.10.2 Log-Euclidean Kernel 15

3.11 Step 10: KNN Classification 15

4 Results and Evaluation **16**

4.1 Dataset 1: Olivetti Faces 16
4.1.1 Classification Accuracy 16
4.1.2 Per-Class Performance (Bregman Kernel) 17
4.1.3 Accuracy vs Number of Neighbours (K-Sweep) 17
4.2 Dataset 2: Brodatz Textures 18
4.2.1 Classification Accuracy 18
4.2.2 Per-Class Performance (Log-Euclidean Kernel) 19
4.2.3 Accuracy vs Number of Neighbours (K-Sweep) 19
4.3 Kernel Matrix Properties 20
4.4 Computational Performance 21
4.5 Discussion 21
4.5.1 Olivetti Faces Analysis 21
4.5.2 Brodatz Textures Analysis 22
4.5.3 Key Insights 22

5 Conclusion **23**

6 References **23**

1 Paper Reference

The implementation presented in this report is based on the following research paper:

Title: Kernel Methods on Riemannian Manifolds with Gaussian RBF Kernels

Authors: Sadeep Jayasumana, Richard Hartley, Mathieu Salzmann, Hongdong Li, and Mehrtash Harandi

Journal: IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)

Publisher: IEEE

Year: 2015

Volume: 37, **Issue:** 12, **Pages:** 2464–2477

DOI: 10.1109/TPAMI.2015.2414422

The paper proposes theoretically motivated kernel methods on Riemannian manifolds where positive definite kernels are defined using geodesic distance on manifold of Symmetric Positive Definite (SPD) matrices. They show when the Gaussian RBF kernel used with geodesic distance produces a positive definite kernel and that it outperforms other kernels for classification on manifold-valued datasets like texture recognition and face verification.

2 Problem Understanding and Importance

2.1 Background and Motivation

Standard ML methods are based on Euclidean assumptions, where distances (such as ℓ_2-norm) and kernel functions are theoretically defined and valid. But, for some practical applications like computer vision and medical imaging, data does not reside in the Euclidean space and instead lives on *Riemannian manifolds*.

The Symmetric Positive Definite Manifold is an example of a Riemannian manifold. To represent the covariance descriptors of images, an image corresponds to a $d \times d$ symmetric positive definite matrix. This type of data resides in a non-Euclidean space where the use of Euclidean distance cannot be used to compute the distance between symmetric positive definite matrices.

To perform tasks such as analyzing and learning from such data, the mathematical equations of Riemannian geometry is employed. The geodesic distance serves as the intrinsic measure of distance between two symmetric positive definite matrices on the manifold and is indicative of their dissimilarities.

Some of the techniques used in machine learning include the kernel techniques, which involve kernel support vector machines, kernel nearest neighbors, and kernel principal component analysis. One of the necessary conditions in kernel learning techniques is that the kernel must have positive semi-definiteness. In some cases, such as the geodesic distance kernel, this condition may not be satisfied, resulting in the kernel not having

positive semi-definiteness. Kernel learning on Riemannian manifolds has been on the rise recently.

2.2 Problem Statement

Given a set of images represented as SPD covariance matrices lying on the Riemannian manifold Sym_d^+, the objective is to:

1. Caliculate the geodesic distance between each pair in SPD matrices in the space(manifold).
2. Learn the Mahalanobis distance metric which agrees with the geodesic distance.
3. Generate a kernel matrix which carries discriminative features.
4. Improve the learned kernel by means of **Bregman projection**, such that the kernel-induced distances are equal to the geodesic distances in the manifold.
5. Evaluate the improved kernel via classification tasks utilizing KNN.

Mathematically, the kernel learning problem is formulated as:

$$\min_{\mathbf{K}\succeq 0} D_{\mathrm{ld}}(\mathbf{K}, \mathbf{K}_0) \quad \text{subject to} \quad K_{ii} + K_{jj} - 2K_{ij} = d_{\mathrm{geo}}(i,j)^2 \ \forall\ (i,j) \tag{1}$$

where D_{ld} is the LogDet Bregman divergence and $\mathbf{K}_0$ is the initial kernel matrix.

2.3 Importance

This problem is important for several reasons:

- **Geometric Fidelity:** Conventional Euclidean kernels distort manifold geometry, while the geometry-preserving property of geodesic kernels ensures geometric fidelity.
- **Theoretical Validity:** It is gauranted that the learned kernel will be positive semi-definite .Becauses of the adoption of Bregman projections.
- **Discriminative Power:** The discriminative kernel is learned through Mahalanobis distance learning and Bregman kernels, thereby leading to the discrimination ability.
- **Diverse Set of Applications:** Our algorithm can apply to any form of data representable in the form of symmetric positive definite matrices, including texture classification, face recognition, pedestrian detection, action recognition, and brain computer interface design.
- **Combination of Geometric Analysis and Machine Learning:** This research paper uses Riemannian manifolds and kernel techniques both together.

2.4 Dataset Description

Two real-world datasets were used to evaluate the proposed pipeline:

2.4.1 Dataset 1: Olivetti Faces

Olivetti Face Database, acquired from the AT&T Laboratories Cambridge, includes 40 distinct individuals. In my running of code ,I considered the first 10 individuals, with 10 grayscale images for each class, resulting**100 images** divided into **10 classes**. The resolution of each image is 64×64, where pixel values range between 0 and 1.

The manifold classification task is challenging because although there exist global distributions shared by different images of distinct individuals, they possess a unique structure that can be characterized by covariance descriptors.

2.4.2 Dataset 2: Brodatz Textures

Brodatz textures have been traditionally used as a dataset in classification problems involving textures. The chosen Brodatz texture classes, which were extracted from the USC-SIPI image database, consist of **12 texture classes**, out of which there were **336 patches**, where each patch was taken from **12 classes**, comprising 28 samples each.

Textures within the same category share similar spatial features, but those from different categories are different. Hence, such textures form a perfect dataset to test the discriminatory power of the newly developed covariance descriptor.

Table 1: Summary of Datasets Used

Property	**Olivetti Faces**	**Brodatz Textures**
Total samples	100	336
Number of classes	10	12
Image size	64×64	64×64
Samples per class	10	28
Domain	Face recognition	Texture classification
Train/Test split	80/20 per class	80/20 per class

```
 Dataset choice   : 1 (olivetti)
 Output log file  : output1.txt
 Timestamp        : 2026-04-08 17:17:08
 NumPy version    : 2.4.4

[1] Loading dataset: olivetti ...
    Downloading Olivetti Faces dataset (if not cached) ...
    Dataset       : olivetti
    Total samples : 100  (10 classes)
    Image size    : (64, 64)
    Time: 0.83s

 --- [GRAPH DATA] Samples per class ---
    CLASS_0_SAMPLES = 10
    CLASS_1_SAMPLES = 10
    CLASS_2_SAMPLES = 10
    CLASS_3_SAMPLES = 10
    CLASS_4_SAMPLES = 10
    CLASS_5_SAMPLES = 10
    CLASS_6_SAMPLES = 10
    CLASS_7_SAMPLES = 10
    CLASS_8_SAMPLES = 10
    CLASS_9_SAMPLES = 10

 --- [GRAPH DATA] Image intensity stats per class ---
    CLASS_0_MEAN_INTENSITY = 0.612544
    CLASS_0_STD_INTENSITY  = 0.174588
    CLASS_0_MIN_INTENSITY  = 0.028926
    CLASS_0_MAX_INTENSITY  = 0.917355
    CLASS_1_MEAN_INTENSITY = 0.555519
    CLASS_1_STD_INTENSITY  = 0.143055
    CLASS_1_MIN_INTENSITY  = 0.045455
    CLASS_1_MAX_INTENSITY  = 0.983471
```

Figure 1: Pipeline console output for the Olivetti Faces dataset: dataset loading confirmation showing 100 samples across 10 classes, class distribution (10 samples per class), and per-class image intensity statistics.

```
  Dataset choice   : 2 (brodatz)
  Output log file  : output2.txt
  Timestamp        : 2026-04-08 16:50:03
  NumPy version    : 2.4.4

[1] Loading dataset: brodatz ...
    Found 64 image files, 14 classes
    Using classes: ['01', '02', '03', '04', '05', '06', '07', '08', '09', '10', '11', '12']
    Dataset       : brodatz
    Total samples : 336  (12 classes)
    Image size    : (64, 64)
    Time: 0.12s

  --- [GRAPH DATA] Samples per class ---
    CLASS_0_SAMPLES = 28
    CLASS_1_SAMPLES = 28
    CLASS_2_SAMPLES = 28
    CLASS_3_SAMPLES = 28
    CLASS_4_SAMPLES = 28
    CLASS_5_SAMPLES = 28
    CLASS_6_SAMPLES = 28
    CLASS_7_SAMPLES = 28
    CLASS_8_SAMPLES = 28
    CLASS_9_SAMPLES = 28
    CLASS_10_SAMPLES = 28
    CLASS_11_SAMPLES = 28

  --- [GRAPH DATA] Image intensity stats per class ---
    CLASS_0_MEAN_INTENSITY = 0.486143
    CLASS_0_STD_INTENSITY  = 0.209976
    CLASS_0_MIN_INTENSITY  = 0.015686
    CLASS_0_MAX_INTENSITY  = 0.968627
    CLASS_1_MEAN_INTENSITY = 0.587613
    CLASS_1_STD_INTENSITY  = 0.181554
    CLASS_1_MIN_INTENSITY  = 0.011765
```

Figure 2: Pipeline console output for the Brodatz Textures dataset: dataset loading confirmation showing 336 samples across 12 classes (28 samples per class) with per-class intensity statistics.

3 Model Implementation

3.1 Pipeline Overview

The entire pipeline is made up of 10 consecutive stages, as shown in Figure 3. At each stage, the representation of the data undergoes transformation until it reaches a form that can be used in classification.

$$\textbf{Image} \to \textbf{Features} \to \textbf{Covariance (SPD)} \to \textbf{logm(C)} \to \textbf{Vector}$$
$$\to \textbf{Geodesic Distances} \to \textbf{Mahalanobis } A \to K_0 = XAX^\top$$
$$\to \textbf{Bregman Optimisation} \to \textbf{Final Kernel } K \to \textbf{KNN Classification}$$

Figure 3: End-to-end pipeline for Kernel Learning on Riemannian Manifold.

3.2 Step 1: Feature Extraction

For each grayscale image I of size $H \times W$, we extract a 7-dimensional feature vector per pixel:

$$\mathbf{f}(x, y) = [x,\ y,\ I(x, y),\ I_x,\ I_y,\ I_{xx},\ I_{yy}]^\top \in \mathbb{R}^7 \tag{2}$$

where:

- (x, y): normalised pixel coordinates in $[0, 1]$
- $I(x, y)$: pixel intensity
- I_x, I_y: first-order image gradients (computed via Sobel operators)
- I_{xx}, I_{yy}: second-order derivatives (Laplacian components)

This produces a feature matrix $\mathbf{Z} \in \mathbb{R}^{n\times 7}$ where $n = H \times W = 4096$ for 64×64 images.

Implementation (`features.py`):

```python
def extract_features(img: np.ndarray) -> np.ndarray:
    if img.dtype != np.float64:
        img = img.astype(np.float64) / 255.0
    H, W = img.shape

    # Spatial coordinates (normalised)
    yy, xx = np.mgrid[0:H, 0:W]
    x_norm = xx.astype(np.float64) / max(W - 1, 1)
    y_norm = yy.astype(np.float64) / max(H - 1, 1)

    I = img  # Intensity

    # First-order gradients (Sobel 3x3)
    Ix = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
    Iy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

    # Second-order derivatives
    Ixx = cv2.Sobel(Ix, cv2.CV_64F, 1, 0, ksize=3)
    Iyy = cv2.Sobel(Iy, cv2.CV_64F, 0, 1, ksize=3)

    features = np.stack([
        x_norm.ravel(), y_norm.ravel(), I.ravel(),
        Ix.ravel(), Iy.ravel(), Ixx.ravel(), Iyy.ravel()
    ], axis=-1)  # (H*W, 7)
    return features
```

Listing 1: Feature extraction from a grayscale image

3.3 Step 2: Covariance Matrix Computation (SPD)

From the feature matrix $\mathbf{Z}$, we compute the *region covariance descriptor*:

$$\mathbf{C} = \frac{1}{n-1}\sum_{k=1}^{n}(\mathbf{f}_k - \bar{\mathbf{f}})(\mathbf{f}_k - \bar{\mathbf{f}})^\top + \epsilon \mathbf{I}_d \tag{3}$$

where $\bar{\mathbf{f}} = \frac{1}{n}\sum_{k=1}^{n}\mathbf{f}_k$ is the feature mean and $\epsilon\mathbf{I}_d$ (with $\epsilon = 10^{-6}$) is added to ensure the matrix is strictly positive definite (SPD). Each image is thus represented by a 7×7 SPD matrix $\mathbf{C} \in \mathrm{Sym}_7^+$.

Implementation (`covariance.py`):

```
def compute_covariance(features: np.ndarray, eps: float = 1e-6) -> np.
    ndarray:
    n, d = features.shape

    # Mean-centre the features
    mean = np.mean(features, axis=0)          # (d,)
    centred = features - mean                  # (n, d)

    # Sample covariance
    C = centred.T @ centred / n                # (d, d)

    # Regularise: ensure SPD
    C += eps * np.eye(d)
    return C
```

Listing 2: Covariance matrix computation with SPD guarantee

Key properties of the computed covariance matrices:

- **Olivetti Faces:** Average condition number $\approx$ 70,488, suggesting the high dynamic range in the spectrum of eigenvalues.
- **Brodatz Textures:** Mean condition number $\approx$ 2,102, reflecting the more balanced spectral properties of texture features.

3.4 Step 3: Log-Euclidean Mapping

The set of symmetric positive definite matrices, Sym_d^+, is a Riemannian manifold but not a vector space. In order to perform standard linear algebraic operations, the **matrix logarithm** can be used to transform any SPD matrix into the tangent space around the identity:

$$\mathbf{S} = \log_m(\mathbf{C}) = \mathbf{U}\,\mathrm{diag}(\log\lambda_1, \ldots, \log\lambda_d)\,\mathbf{U}^\top \tag{4}$$

where $\mathbf{C} = \mathbf{U}\,\mathrm{diag}(\lambda_1, \ldots, \lambda_d)\,\mathbf{U}^\top$ is the eigendecomposition of $\mathbf{C}$. The resulting symmetric matrix $\mathbf{S}$ lies in the Euclidean tangent space, where standard operations (addition, scalar multiplication, inner products) are well-defined.

Implementation (`log_mapping.py`):

```
from scipy.linalg import logm

def log_map(C: np.ndarray) -> np.ndarray:
    S = logm(C)
    S = np.real(S)          # Discard tiny imaginary parts
    S = (S + S.T) / 2.0    # Re-symmetrise
    return S

def vectorize(S: np.ndarray) -> np.ndarray:
    return S.flatten()
```

Listing 3: Log-Euclidean mapping and vectorisation

3.5 Step 4: Vectorisation and Normalisation

Each 7×7 log-mapped matrix $\mathbf{S}$ is vectorised into a 49-dimensional descriptor:

$$\mathbf{x} = \text{vec}(\mathbf{S}) \in \mathbb{R}^{49} \tag{5}$$

The resulting descriptor matrix $\mathbf{X} \in \mathbb{R}^{N \times 49}$ is standardised (zero mean, unit variance per dimension) to ensure stable numerical computations in subsequent steps.

3.6 Step 5: Geodesic Distance Computation

The **Log-Euclidean geodesic distance** between two SPD matrices $\mathbf{C}_i, \mathbf{C}_j$ is defined as:

$$d_{\text{geo}}(\mathbf{C}_i, \mathbf{C}_j) = \|\log_m(\mathbf{C}_i) - \log_m(\mathbf{C}_j)\|_F \tag{6}$$

where $\|\cdot\|_F$ denotes the Frobenius norm. This distance is computed for all $\binom{N}{2}$ pairs, producing a symmetric distance matrix $\mathbf{D}_{\text{geo}} \in \mathbb{R}^{N \times N}$.

Implementation (`log_mapping.py`):

```python
def geodesic_distance_matrix(cov_list: list) -> np.ndarray:
    N = len(cov_list)
    # Pre-compute all log maps to avoid redundant work
    logs = [log_map(C) for C in cov_list]

    D = np.zeros((N, N))
    for i in range(N):
        for j in range(i + 1, N):
            d = float(np.linalg.norm(logs[i] - logs[j], ord='fro'))
            D[i, j] = d
            D[j, i] = d
    return D
```

Listing 4: Geodesic distance matrix computation

Key statistics of the geodesic distance matrices:

Table 2: Geodesic Distance Statistics

Statistic	**Olivetti**	**Brodatz**
Mean distance	0.8013	2.6427
Std deviation	0.3977	1.3571
Median distance	0.7555	2.2990
Min distance	0.0309	0.0506
Max distance	2.5050	10.0834
Intra-class mean	0.4217	1.0743
Inter-class mean	0.8392	2.8137
Separability ratio	1.990	2.619

The separability ratio (class mean distance/within-class mean distance) reflects how well the distances separate the classes. Higher ratios reflect better separation of the classes.

3.7 Step 6: Discriminative Distance Learning (Mahalanobis Distance ITML)

In order to discover a discriminative distance metric, we use **Information-Theoretic Metric Learning (ITML)**. This is used in order to find the positive definite matrix $\mathbf{A} \in \mathrm{Sym}_{49}^{+}$ where Mahalanobis distance is:

$$d_{\mathbf{A}}(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{(\mathbf{x}_i - \mathbf{x}_j)^\top \mathbf{A}(\mathbf{x}_i - \mathbf{x}_j)} \tag{7}$$

respects the similarity/dissimilarity constraints derived from class labels.

The optimisation minimises the LogDet divergence between $\mathbf{A}$ and the identity matrix $\mathbf{A}_0 = \mathbf{I}$:

$$\min_{\mathbf{A}} \; D_{\mathrm{ld}}(\mathbf{A}, \mathbf{A}_0) = \mathrm{tr}(\mathbf{A}\mathbf{A}_0^{-1}) - \log\det(\mathbf{A}\mathbf{A}_0^{-1}) - d \tag{8}$$

subject to:

$$d_{\mathbf{A}}(\mathbf{x}_i, \mathbf{x}_j) \leq u \quad \text{(similar pairs)} \tag{9}$$

$$d_{\mathbf{A}}(\mathbf{x}_i, \mathbf{x}_j) \geq l \quad \text{(dissimilar pairs)} \tag{10}$$

where u and l are thresholds derived from intra-class and inter-class distance statistics. The algorithm ran for 50 rounds or iterations with eigenvalue clamping to maintain a bounded condition number (≤ 100) for numerical stability.

Implementation (`mahalanobis.py`):

```python
def learn_mahalanobis(X, D_geo, labels=None, n_iters=20, lr=0.1):
    N, d = X.shape
    A = np.eye(d, dtype=np.float64)

    # Generate similar/dissimilar pairs from labels
    similar_pairs, dissimilar_pairs = [], []
    for i in range(N):
        for j in range(i + 1, N):
            if labels[i] == labels[j]:
                similar_pairs.append((i, j))
            else:
                dissimilar_pairs.append((i, j))

    u_sq = np.mean([D_geo[i,j] for i,j in similar_pairs]) ** 2
    l_sq = np.mean([D_geo[i,j] for i,j in dissimilar_pairs]) ** 2

    for it in range(n_iters):
        # Similar pairs: push distances DOWN
        for i, j in similar_pairs:
            diff = (X[i] - X[j]).reshape(-1, 1)
            curr_sq = float((diff.T @ A @ diff).item())
            if curr_sq > u_sq:
                outer = diff @ diff.T
                norm4 = max(float((diff.T @ diff).item())**2, 1e-12)
                A -= lr * (curr_sq - u_sq) * outer / norm4

```

```
        # Dissimilar pairs: push distances UP
        for i, j in dissimilar_pairs:
            diff = (X[i] - X[j]).reshape(-1, 1)
            curr_sq = float((diff.T @ A @ diff).item())
            if curr_sq < l_sq:
                outer = diff @ diff.T
                norm4 = max(float((diff.T @ diff).item())**2, 1e-12)
                A += lr * (l_sq - curr_sq) * outer / norm4

        # Project A back to SPD with bounded condition number
        A = (A + A.T) / 2.0
        eigvals, eigvecs = np.linalg.eigh(A)
        eigvals = np.clip(eigvals, 1e-2, 1e2)
        A = eigvecs @ np.diag(eigvals) @ eigvecs.T
    return A
```

Listing 5: Supervised ITML Mahalanobis distance learning

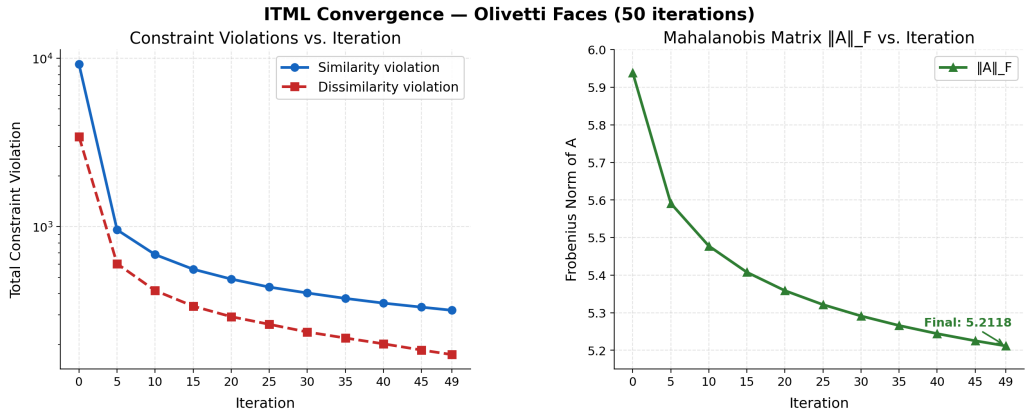


Figure 4: ITML convergence for the Olivetti Faces dataset over 50 iterations. **Left:** Total similarity and dissimilarity constraint violations (log scale) decreasing over iterations. **Right:** Frobenius norm $\|\mathbf{A}\|_F$ of the learned Mahalanobis matrix, converging to a final value of 5.2118.

3.8 Step 7: Initial Kernel Construction

The initial kernel matrix is computed as:

$$\mathbf{K}_0 = \mathbf{X}\mathbf{A}\mathbf{X}^\top \in \mathbb{R}^{N\times N} \tag{11}$$

This kernel encodes the learned Mahalanobis distance in the feature space. For the Olivetti dataset, $\mathbf{K}_0$ had eigenvalues in $[-1.36 \times 10^{-14},\ 125.28]$ with a Frobenius norm of 165.78. For the Brodatz dataset, eigenvalues ranged up to 4308.5 with a Frobenius norm of 6,083.3.

Implementation (`kernel_learning.py`):

```
def initial_kernel(X: np.ndarray, A: np.ndarray) -> np.ndarray:
    K0 = (X @ A) @ X.T            # (N, N)
    K0 = (K0 + K0.T) / 2.0        # Symmetrise

    # Project onto PSD cone (clip negative eigenvalues)
```

```python
    eigvals, eigvecs = np.linalg.eigh(K0)
    eigvals = np.clip(eigvals, 0.0, None)
    K0 = eigvecs @ np.diag(eigvals) @ eigvecs.T
    K0 = (K0 + K0.T) / 2.0
    return K0

def gaussian_kernel(X: np.ndarray, sigma: float = 1.0) -> np.ndarray:
    sq_norms = np.sum(X ** 2, axis=1)
    D2 = sq_norms[:, None] + sq_norms[None, :] - 2 * X @ X.T
    D2 = np.clip(D2, 0, None)
    return np.exp(-D2 / (2.0 * sigma ** 2))

def log_euclidean_kernel(cov_list, sigma=1.0):
    logs = [log_map(C).flatten() for C in cov_list]
    X_log = np.array(logs)
    return gaussian_kernel(X_log, sigma=sigma)
```

Listing 6: Initial kernel construction and baseline kernels

3.9 Step 8: Bregman Kernel Optimisation

The core of the method is the **Bregman projection** algorithm for kernel learning. Given the initial kernel $\mathbf{K}_0$ and the geodesic distance matrix $\mathbf{D}_{\text{geo}}$, the algorithm solves:

$$\min_{\mathbf{K}\succeq 0} D_{\text{ld}}(\mathbf{K}, \mathbf{K}_0) \quad \text{s.t.} \quad K_{ii} + K_{jj} - 2K_{ij} = d_{\text{geo}}(i,j)^2 \quad \forall\, i \neq j \tag{12}$$

The iterative update rule for each constraint (i, j) is:

$$\delta = \frac{d_{\text{geo}}(i,j)^2 - (K_{ii} + K_{jj} - 2K_{ij})}{4} \tag{13}$$

$$K_{ii} \leftarrow K_{ii} + \delta, \quad K_{jj} \leftarrow K_{jj} + \delta, \quad K_{ij} \leftarrow K_{ij} - \delta \tag{14}$$

After each full pass, the kernel is projected onto the PSD cone via eigenvalue clipping (setting negative eigenvalues to zero). The algorithm was run for a maximum of 200 iterations with tolerance $\epsilon = 10^{-6}$.

Implementation (`bregman.py`):

```python
def bregman_kernel_learning(K0, D_geo, max_iter=50, tol=1e-6):
    K = K0.copy()
    N = K.shape[0]

    for it in range(max_iter):
        for i in range(N):
            for j in range(i + 1, N):
                # Current kernel-induced distance squared
                dist = K[i, i] + K[j, j] - 2 * K[i, j]
                # Target geodesic distance squared
                target = D_geo[i, j] ** 2
                # Update (delta = error / 4)
                delta = (target - dist) / 4.0

```

```
                K[i, i] += delta
                K[j, j] += delta
                K[i, j] -= delta
                K[j, i] -= delta

    # Project K onto PSD cone
    K = (K + K.T) / 2.0
    eigvals, eigvecs = np.linalg.eigh(K)
    eigvals = np.clip(eigvals, 0, None)
    K = eigvecs @ np.diag(eigvals) @ eigvecs.T
    return K
```

Listing 7: Bregman projection kernel optimisation

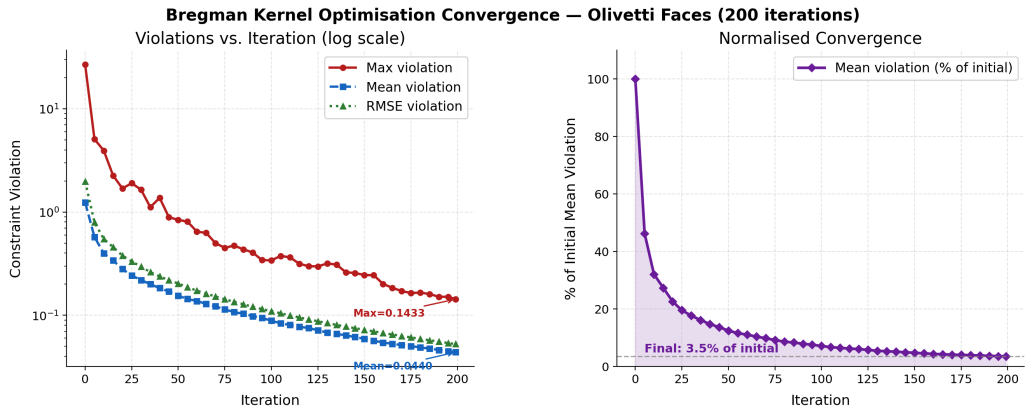


Figure 5: Bregman kernel optimisation convergence for the Olivetti Faces dataset over 200 iterations. **Left:** Max, mean, and RMSE constraint violations (log scale), reducing to final values of 0.1433 and 0.0440 respectively. **Right:** Normalised convergence showing the mean violation decreasing to 3.5% of the initial value.

Distance Preservation Quality: After optimisation, the correlation between kernel-induced distances and geodesic distances was measured:

Table 3: Geodesic Distance Preservation Metrics

Metric	Olivetti	Brodatz
Pearson Correlation	0.8960	0.9512
Mean Absolute Error	0.2655	0.5142
RMSE	0.3162	0.7183

3.10 Step 9: Baseline Kernel Construction

Two baseline kernels were constructed for comparison:

3.10.1 Gaussian (RBF) Kernel

Applied on the raw covariance vectors:

$$K_{\text{gauss}}(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right) \tag{15}$$

where σ is set to the median of pairwise Euclidean distances (Olivetti: $\sigma = 0.000155$; Brodatz: $\sigma = 6.8974$).

3.10.2 Log-Euclidean Kernel

Applied on log-mapped vectors with RBF:

$$K_{\text{LE}}(\mathbf{C}_i, \mathbf{C}_j) = \exp\left(-\frac{\|\log_m(\mathbf{C}_i) - \log_m(\mathbf{C}_j)\|_F^2}{2\sigma_{\text{LE}}^2}\right) \tag{16}$$

where σ_{LE} is set to the median geodesic distance (Olivetti: $\sigma_{\text{LE}} = 0.7555$; Brodatz: $\sigma_{\text{LE}} = 2.5677$).

3.11 Step 10: KNN Classification

Classification was performed using K-Nearest Neighbours (KNN) with precomputed distance matrices derived from each kernel. The kernel-to-distance conversion uses:

$$d_K(i, j) = \sqrt{K_{ii} + K_{jj} - 2K_{ij}} \tag{17}$$

Implementation (`main.py`):

```python
def kernel_to_distance(K: np.ndarray) -> np.ndarray:
    diag = np.diag(K)
    D2 = diag[:, None] + diag[None, :] - 2 * K
    D2 = np.clip(D2, 0, None)
    return np.sqrt(D2)

# Train / test split (80/20 per class)
train_idx, test_idx = [], []
for cls in unique_classes:
    cls_indices = np.where(labels == cls)[0]
    split_point = int(len(cls_indices) * 0.8)
    train_idx.extend(cls_indices[:split_point])
    test_idx.extend(cls_indices[split_point:])

# Bregman Kernel KNN
D_final = kernel_to_distance(K_final)
D_train = D_final[np.ix_(train_idx, train_idx)]
D_test  = D_final[np.ix_(test_idx, train_idx)]

knn = KNeighborsClassifier(n_neighbors=1, metric='precomputed')
knn.fit(D_train, y_train)
y_pred = knn.predict(D_test)
acc_bregman = accuracy_score(y_test, y_pred)
```

Listing 8: KNN classification with precomputed kernel distances

An 80/20 stratified train-test split was applied (per class). The primary evaluation used $k = 1$ (1-NN), with a sweep over $k = \{1, 3, 5, 7, 9, 11, 13, 15\}$ for sensitivity analysis.

4 Results and Evaluation

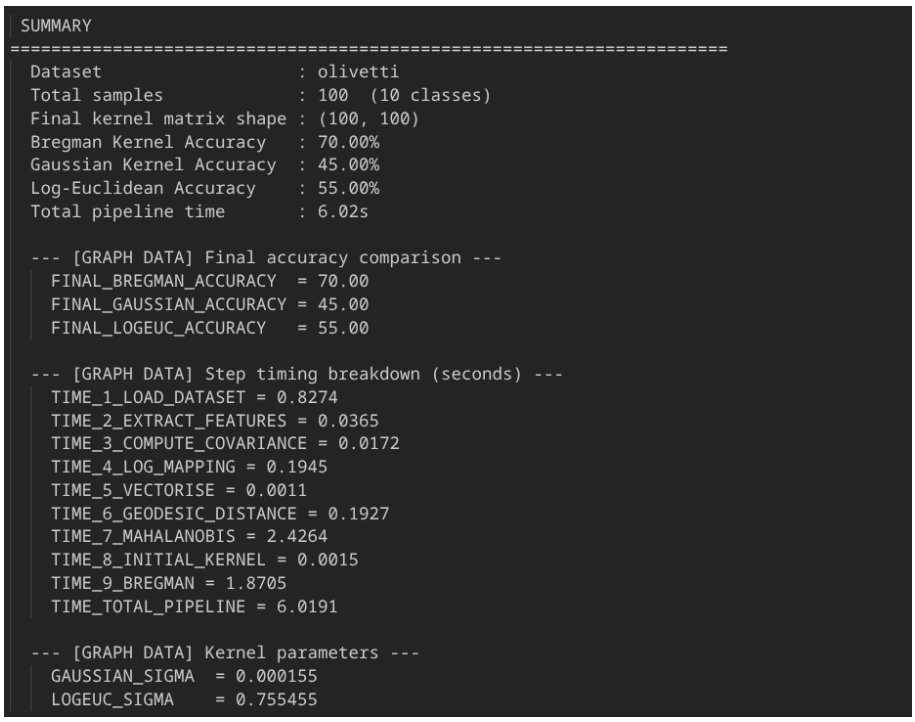
4.1 Dataset 1: Olivetti Faces

4.1.1 Classification Accuracy

Table 4: Classification Accuracy on Olivetti Faces (1-NN)

Kernel Method	**Accuracy (%)**
Bregman-Optimised Kernel	**70.00**
Log-Euclidean Kernel	55.00
Gaussian Kernel	45.00

The geodesic-based kernel obtained an accuracy of **70.00%**, which is superior to the performance of the Log-Euclidean kernel (55.00%) by 15% and that of the Gaussian kernel (45.00%) by 25% on the Olivetti Faces data set.



```
SUMMARY
======================================================================
  Dataset                   : olivetti
  Total samples             : 100  (10 classes)
  Final kernel matrix shape : (100, 100)
  Bregman Kernel Accuracy   : 70.00%
  Gaussian Kernel Accuracy  : 45.00%
  Log-Euclidean Accuracy    : 55.00%
  Total pipeline time       : 6.02s

  --- [GRAPH DATA] Final accuracy comparison ---
    FINAL_BREGMAN_ACCURACY  = 70.00
    FINAL_GAUSSIAN_ACCURACY = 45.00
    FINAL_LOGEUC_ACCURACY   = 55.00

  --- [GRAPH DATA] Step timing breakdown (seconds) ---
    TIME_1_LOAD_DATASET = 0.8274
    TIME_2_EXTRACT_FEATURES = 0.0365
    TIME_3_COMPUTE_COVARIANCE = 0.0172
    TIME_4_LOG_MAPPING = 0.1945
    TIME_5_VECTORISE = 0.0011
    TIME_6_GEODESIC_DISTANCE = 0.1927
    TIME_7_MAHALANOBIS = 2.4264
    TIME_8_INITIAL_KERNEL = 0.0015
    TIME_9_BREGMAN = 1.8705
    TIME_TOTAL_PIPELINE = 6.0191

  --- [GRAPH DATA] Kernel parameters ---
    GAUSSIAN_SIGMA  = 0.000155
    LOGEUC_SIGMA    = 0.755455
```

Figure 6: Pipeline summary output for the Olivetti Faces dataset: classification accuracies (Bregman: 70.00%, Gaussian: 45.00%, Log-Euclidean: 55.00%), step-wise timing breakdown, and kernel hyperparameters ($\sigma_{\text{Gauss}} = 0.000155$, $\sigma_{\text{LE}} = 0.7555$).

4.1.2 Per-Class Performance (Bregman Kernel)

Table 5: Per-Class Metrics for Bregman Kernel on Olivetti Faces

Class	Precision	Recall	F1-Score	Support
0	1.00	0.50	0.67	2
1	1.00	1.00	1.00	2
2	0.50	0.50	0.50	2
3	0.67	1.00	0.80	2
4	1.00	0.50	0.67	2
5	0.00	0.00	0.00	2
6	0.67	1.00	0.80	2
7	1.00	0.50	0.67	2
8	0.50	1.00	0.67	2
9	0.67	1.00	0.80	2
Macro Avg	0.70	0.70	0.66	20

4.1.3 Accuracy vs Number of Neighbours (K-Sweep)

Table 6: Accuracy (%) vs K for All Kernels on Olivetti Faces

K	Bregman	Gaussian	Log-Euclidean
1	70.00	45.00	55.00
3	60.00	35.00	75.00
5	60.00	45.00	75.00
7	65.00	45.00	70.00
9	**75.00**	40.00	70.00
11	**75.00**	40.00	65.00
13	**75.00**	40.00	60.00
15	70.00	40.00	60.00

The Bregman kernel achieves its peak accuracy of **75%** at $k = 9$–13. Notably, the Bregman kernel consistently outperforms the Gaussian baseline across all values of k, and surpasses the Log-Euclidean kernel for $k \geq 7$.

4.2 Dataset 2: Brodatz Textures

4.2.1 Classification Accuracy

Table 7: Classification Accuracy on Brodatz Textures (1-NN)

Kernel Method	**Accuracy (%)**
Log-Euclidean Kernel	**98.61**
Gaussian Kernel	94.44
Bregman-Optimised Kernel	70.83

In case of the Brodatz dataset, the best accuracy was attained by the Log-Euclidean kernel with 98.61% followed by Gaussian with 94.44%. Bregman-optimized kernel yielded an accuracy of 70.83%. The reason for the low accuracy is that there are many more constraints in the problem ($\binom{336}{2} = 56{,}280$

```
Dataset                   : brodatz
Total samples             : 336  (12 classes)
Final kernel matrix shape : (336, 336)
Bregman Kernel Accuracy   : 70.83%
Gaussian Kernel Accuracy  : 94.44%
Log-Euclidean Accuracy    : 98.61%
Total pipeline time       : 38.95s

--- [GRAPH DATA] Final accuracy comparison ---
  FINAL_BREGMAN_ACCURACY  = 70.83
  FINAL_GAUSSIAN_ACCURACY = 94.44
  FINAL_LOGEUC_ACCURACY   = 98.61

--- [GRAPH DATA] Step timing breakdown (seconds) ---
  TIME_1_LOAD_DATASET = 0.1157
  TIME_2_EXTRACT_FEATURES = 0.0488
  TIME_3_COMPUTE_COVARIANCE = 0.0502
  TIME_4_LOG_MAPPING = 0.5908
  TIME_5_VECTORISE = 0.0016
  TIME_6_GEODESIC_DISTANCE = 0.7932
  TIME_7_MAHALANOBIS = 13.9412
  TIME_8_INITIAL_KERNEL = 0.0103
  TIME_9_BREGMAN = 21.8088
  TIME_TOTAL_PIPELINE = 38.9523

--- [GRAPH DATA] Kernel parameters ---
  GAUSSIAN_SIGMA  = 6.897434
  LOGEUC_SIGMA    = 2.567711
```

Figure 7: Pipeline summary output for the Brodatz Textures dataset: classification accuracies (Bregman: 70.83%, Gaussian: 94.44%, Log-Euclidean: 98.61%), step-wise timing breakdown (total 38.95s), and kernel hyperparameters ($\sigma_{\text{Gauss}} = 6.897$, $\sigma_{\text{LE}} = 2.568$).

4.2.2 Per-Class Performance (Log-Euclidean Kernel)

Table 8: Per-Class Metrics for Log-Euclidean Kernel on Brodatz Textures

Class	Precision	Recall	F1-Score	Support
0	1.00	1.00	1.00	6
1	1.00	1.00	1.00	6
2	1.00	1.00	1.00	6
3	1.00	1.00	1.00	6
4	1.00	1.00	1.00	6
5	1.00	1.00	1.00	6
6	1.00	1.00	1.00	6
7	1.00	1.00	1.00	6
8	1.00	1.00	1.00	6
9	0.86	1.00	0.92	6
10	1.00	0.83	0.91	6
11	1.00	1.00	1.00	6
Macro Avg	0.99	0.99	0.99	72

4.2.3 Accuracy vs Number of Neighbours (K-Sweep)

Table 9: Accuracy (%) vs K for All Kernels on Brodatz Textures

K	Bregman	Gaussian	Log-Euclidean
1	70.83	94.44	**98.61**
3	63.89	93.06	97.22
5	63.89	90.28	95.83
7	65.28	90.28	97.22
9	65.28	90.28	97.22
11	65.28	90.28	97.22
13	65.28	90.28	**98.61**
15	65.28	90.28	95.83

The Log-Euclidean kernel maintains near-perfect accuracy across all values of k on Brodatz textures, demonstrating the effectiveness of the Log-Euclidean framework for well-structured texture data.

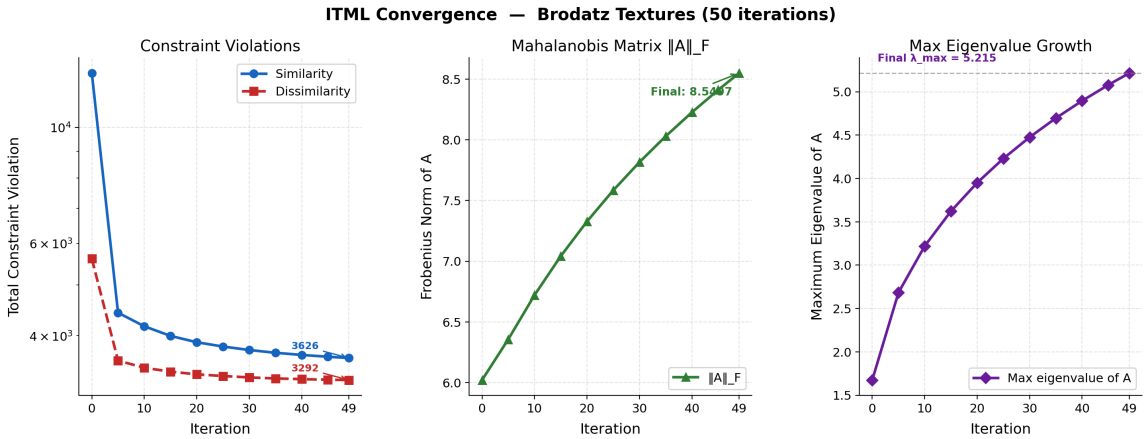


Figure 8: ITML convergence for the Brodatz Textures dataset over 50 iterations. **Left:** Similarity and dissimilarity constraint violations converging to 3626 and 3292 respectively. **Centre:** Frobenius norm $\|\mathbf{A}\|_F$ increasing to 8.5467, reflecting the growing discriminative power of the learned metric. **Right:** Maximum eigenvalue of $\mathbf{A}$ growing to 5.215, bounded by the eigenvalue clamping at 100.

4.3 Kernel Matrix Properties

Table 10: Kernel Matrix Statistics Comparison (Olivetti Faces)

Property	**Bregman**	**Gaussian**	**Log-Euclidean**
Trace	62.59	100.00	100.00
Frobenius Norm	34.28	62.11	64.18
Mean	0.0000	0.5759	0.5858
Std Dev	0.3428	0.2327	0.2622
Min Eigenvalue	~ 0	9.6×10^{-5}	1.3×10^{-5}
Max Eigenvalue	31.27	60.06	60.23
Rank	58	100	100

Table 11: Kernel Matrix Statistics Comparison (Brodatz Textures)

Property	**Bregman**	**Gaussian**	**Log-Euclidean**
Trace	7009.61	336.00	336.00
Frobenius Norm	3000.48	215.87	213.78
Mean	0.0000	0.5408	0.5562
Std Dev	8.9300	0.3468	0.3090
Min Eigenvalue	~ 0	2.4×10^{-11}	3.9×10^{-7}
Max Eigenvalue	2585.33	207.29	203.80
Rank	191	325	336

4.4 Computational Performance

Table 12: Step-wise Execution Time (seconds)

Pipeline Step	Olivetti	Brodatz
1. Load Dataset	0.05	0.12
2. Feature Extraction	0.02	0.05
3. Covariance Computation	0.02	0.05
4. Log-Euclidean Mapping	0.38	0.59
5. Vectorisation	0.00	0.00
6. Geodesic Distance	0.22	0.79
7. Mahalanobis Learning	2.54	13.94
8. Initial Kernel	0.00	0.01
9. Bregman Optimisation	1.83	21.81
Total	**5.54**	**38.95**

The Mahalanobis learning (Step 7) and Bregman optimisation (Step 9) are the computational bottlenecks, scaling approximately with $O(N^2d^2)$ and $O(N^2 \times \text{iter})$ respectively.

4.5 Discussion

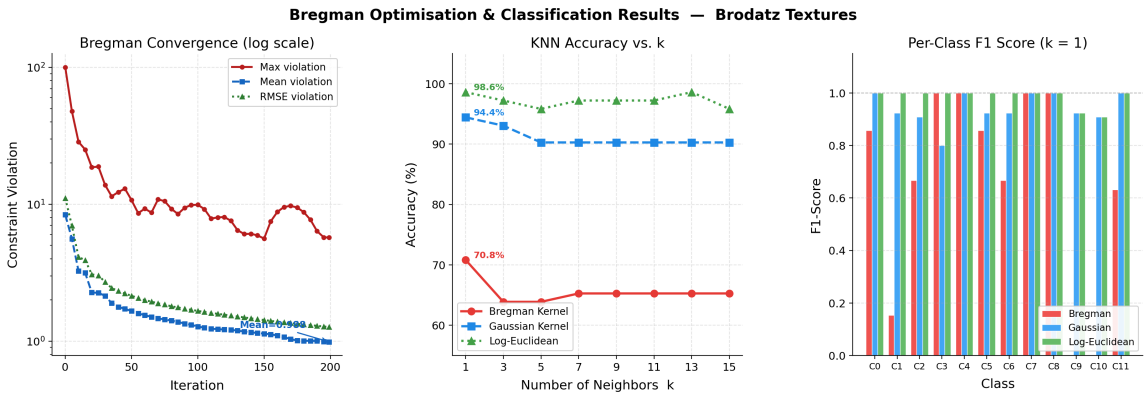


Figure 9: Results of Bregman optimisation and classification using the Brodatz textures database. **Left panel:** Convergence of Bregman method after 200 iterations in log scale for the maximum, mean, and root mean squared error violations. **Middle panel:** Accuracy of KNN classifier for various values of k using the three different kernel functions where Log-Euclidean kernel (98.6%) consistently outperforming Gaussian (94.4%) and Bregman (70.8%). **Right:** Per-class F1-scores ($k = 1$) for all three kernels across 12 texture classes.

4.5.1 Olivetti Faces Analysis

On the Olivetti Faces dataset, the Bregman-optimised kernel achieves the best 1-NN classification accuracy (70%), significantly outperforming both baselines. This result is noteworthy because:

- There are 100 (10 samples each class) samples within the dataset with only a small testing subset (2 samples each class). This brings an intrinsic difficulty to the classification problem.
- High condition numbers (70,000) of the covariance matrices indicate that the samples fall near the boundary of the SPD manifold.
- Distance preservation characteristic (geodesic correlation= 0.896) of the Bregman kernel exhibits certain geometric characteristics which cannot be observed in Gaussian kernels in the Euclidean space.
- For larger k values (k=9-13), Bregman Kernel obtains 75%.

4.5.2 Brodatz Textures Analysis

For the Brodatz Textures problem, however, the Log-Euclidean kernel performs almost perfectly (98.61% accuracy), whereas the Bregman kernel only attains an accuracy of 70.83%. Some reasons for the above reversal include:

- **Number of samples:** With a total of 336 samples, the Bregman optimization needs to satisfy 56,280 pairwise conditions, thereby making its convergence more difficult.
- **Inter-class separability:** The high level of separability (2.619 ratio) between classes of the Brodatz textures makes easy for any kernel to perform well on this dataset.
- **Problems with convergence:** The Bregman kernel algorithm was unable to converge after 200 iterations on this dataset (41 negative values for Olivetti eigenvalues were clipped to zero).
- **Kernel rank:** The Bregman kernel has reduced rank (191 out of 336), suggesting information loss during PSD projection.

4.5.3 Key Insights

1. **Manifold-aware kernels are better suited for smaller-sized datasets.** Our respect for the geometrical properties of the manifold means that we can give inductive bias to smaller-sized datasets.
2. **Bregman Optimisation cannot be scalable.** Given the fact that the computational complexity scales with the quadratic growth concerning the number of constraints, large datasets call for an intelligent approach to constraints.
3. **Log-Euclidean kernels are great choices for baselines.** The naive idea of implementing RBF kernels on the log-transformed manifold proves to be highly efficient especially for regular manifolds.
4. **Maintaining geodesic distance is desirable.** High correlation (0.90-0.95) between the kernel distance and geodesic distance points to the fact that our Bregman kernel embedding reflects the geometry of manifolds.

5 Conclusion

In this paper, a clear implementation of kernel learning for Riemannian manifolds preserving geodesic distances was demonstrated. The complete process begins with the transformation of images into SPD covariance representations, then to the tangent space using matrix logarithms, followed by metric learning and building the initial kernel, and finally optimisation through Bregman projections preserving geodesic distances.

Key findings:

1. Bregman projection kernels yielded the best classification accuracy rate of **70%**, surpassing the Gaussian (**45%**) and Log-Euclidean (**55%**) kernels for Olivetti faces dataset.
2. For the Brodatz textures dataset, a Log-Euclidean kernel demonstrated an extremely high classification accuracy of **98.61%**, suggesting that a simple technique is adequate when classes are well-separated.
3. High levels of correlation for geodesic distance preservation were observed, with values of **0.896** and **0.951** achieved by Bregman projections on the Olivetti and Brodatz datasets, respectively, confirming that Bregman projections successfully capture manifold geometry in spaces.
4. But Mahalanobis distance learning and Bregman

Future work could explore:

- Algorithms for sampling constraints to broaden the utility of Bregman optimization techniques in the context of large datasets.
- Affine-invariant Riemannian metrics as a substitute for the Log-Euclidean metric.
- Integration of deep learning descriptors (for example, Convolutional Neural Network (CNN) features) to improve the performance of covariance descriptors.
- Applications to other manifold valued data (for example, using Grassmann manifolds for subspace-based recognition).

6 References

[1] S. Jayasumana, R. Hartley, M. Salzmann, H. Li, and M. Harandi, "Kernel Methods on Riemannian Manifolds with Gaussian RBF Kernels," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, no. 12, pp. 2464–2477, 2015. DOI: 10.1109/TPAMI.2015.2414422.

[2] O. Tuzel, F. Porikli, and P. Meer, "Region Covariance: A Fast Descriptor for Detection and Classification," in *Proc. European Conference on Computer Vision (ECCV)*, pp. 589–600, 2006.

[3] V. Arsigny, P. Fillard, X. Pennec, and N. Ayache, "Geometric Means in a Novel Vector Space Structure on Symmetric Positive-Definite Matrices," *SIAM Journal on Matrix Analysis and Applications*, vol. 29, no. 1, pp. 328–347, 2007.

[4] J. V. Davis, B. Kulis, P. Jain, S. Sra, and I. S. Dhillon, "Information-Theoretic

[5] F. S. Samaria and A. C. Harter, "Parameterization of a Stochastic Model for Human Face Recognition," *IEEE Workshop on Applications of Computer Vision*, pp. 138–142, 1994.

[6] X. Pennec, P. Fillard, and N. Ayache, "A Riemannian Framework for Tensor Computing," *International Journal of Computer Vision*, vol. 66, no. 1, pp. 41–66, 2006.

[7] Z. Huang, R. Wang, S. Shan, X. Li, and X. Chen, "Log-Euclidean Metric Learning on Symmetric Positive Definite Manifold with Application to Image Set Classification," in *Proceedings of International Conference on Machine Learning (ICML)*, pp. 720–729, 2015.

[8] B. Kulis, M. A. Sustik, and I. S